De la Rosa Valdés Itztli Fernanda

Bazaldua Montoya Jose Juan

Practica: Esquema Conversacional (HTML & JS)

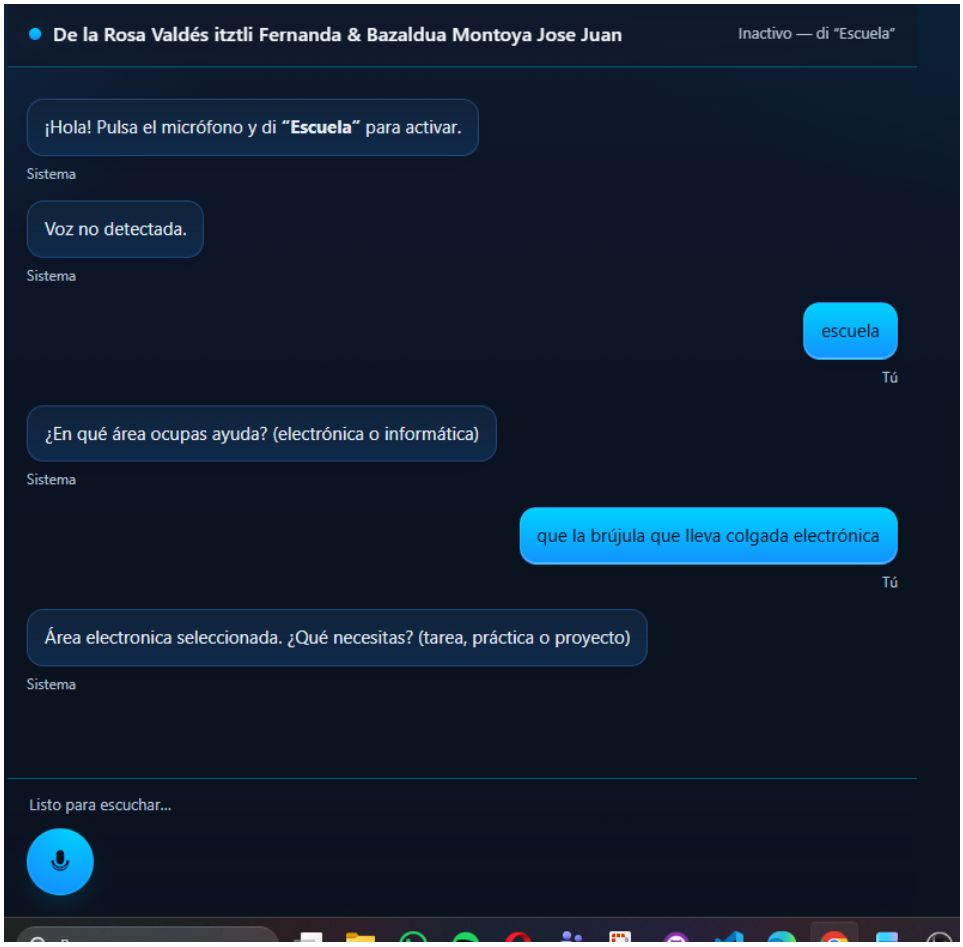
Multimedia 3TM2

Basándonos en el primer avance entregado previamente acerca del esquema conversacional.

Esta práctica implementa un asistente conversacional que combina **STT (Speech-to-Text)** y **TTS (Text-to-Speech)** para permitir interacción por voz con flujo similar a un chatbot. El navegador convierte la voz del usuario en texto mediante la Web Speech API, el sistema interpreta ese texto mediante reglas simples de palabras clave (área y necesidad), genera una respuesta y luego la

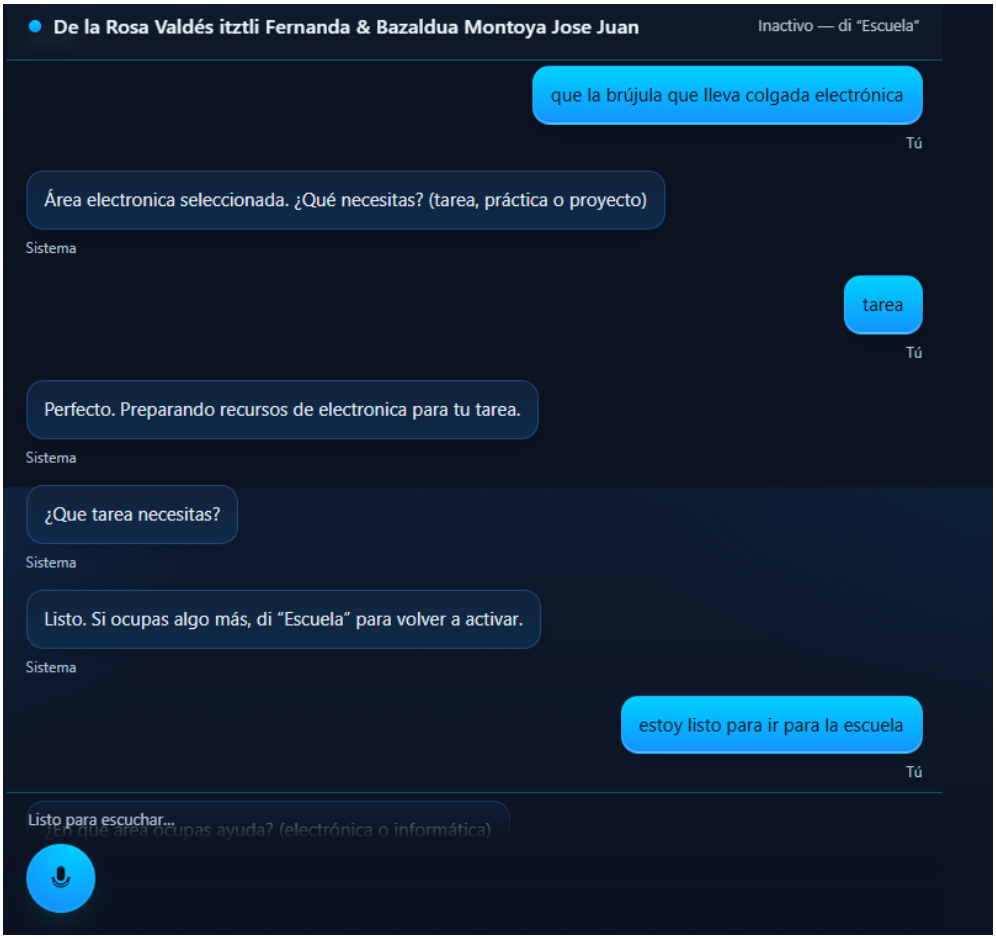
reproduce usando síntesis de voz. De esta forma, el usuario puede mantener un diálogo natural sin necesidad de escribir, y la interfaz muestra cada intercambio como burbujas de chat para reforzar la sensación de conversación.

El flujo comienza en estado **IDLE**, donde el usuario debe decir *"Escuela"* para activar el asistente; al reconocerlo, el sistema pasa a **AWAKE** y solicita el área (electrónica, informática o comunicaciones). Una vez detectada el área, el estado avanza a **AREA**, donde el bot pide la necesidad (tarea, práctica o proyecto). En el caso específico de *informática* + *tarea*, el bot realiza una segunda pregunta para determinar el tema (programación, redes o bases de datos) y responde con definiciones breves mediante TTS. Durante todo el proceso se manejan errores de reconocimiento, comandos globales como "cancelar" y cada mensaje se refleja en la interfaz mediante appendMessage, manteniendo un flujo conversacional simple pero estructurado.

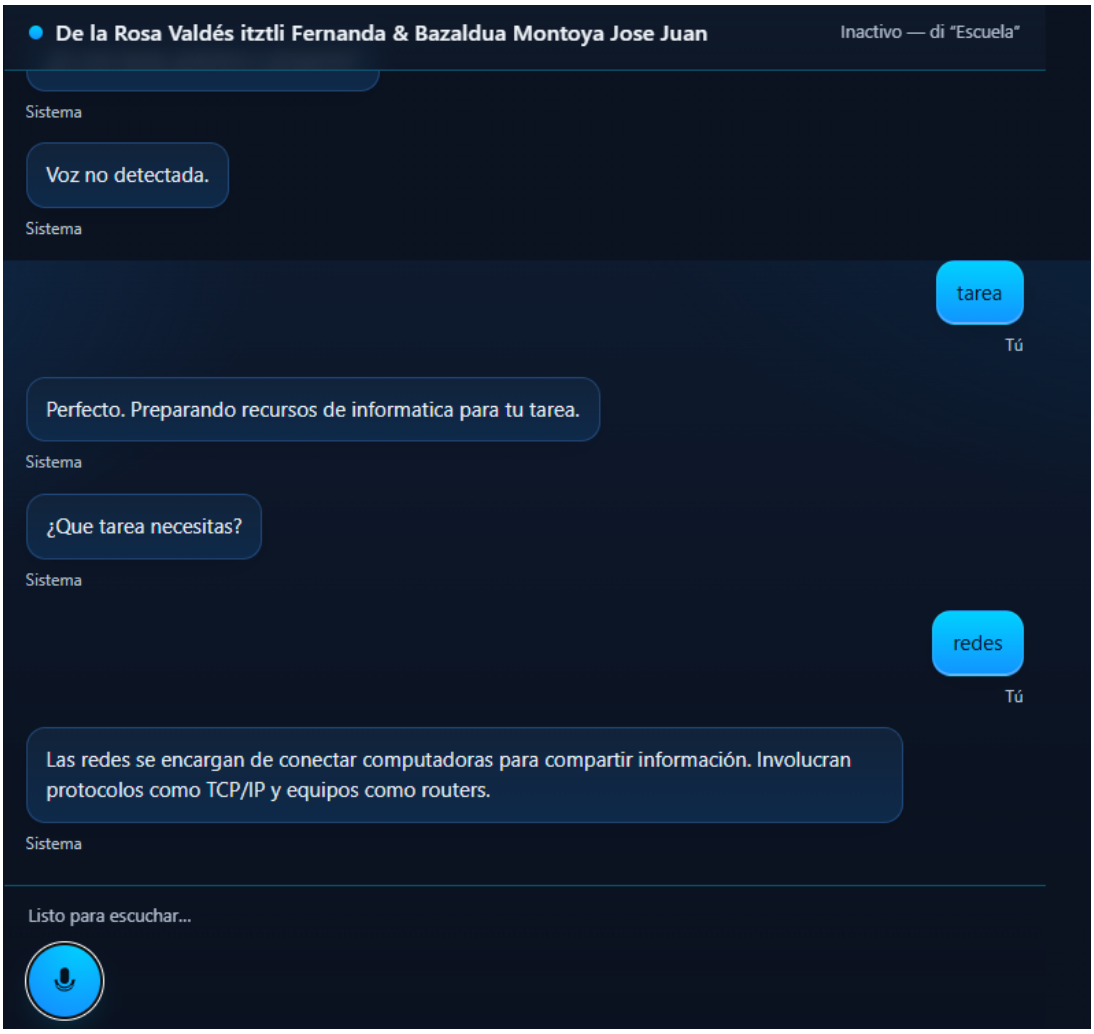


El **HTML** define una interfaz minimalista estructurada en tres secciones principales: un *header* que muestra el estado del asistente, un área central tipo **chat** donde se renderizan los mensajes del usuario y del sistema, y un *footer* que contiene el botón de micrófono para activar la escucha por voz. El documento carga styles.css, integra el script principal app.js y arranca con un mensaje inicial que orienta al usuario sobre cómo activar el asistente mediante el comando "Escuela".

El **CSS** implementa un diseño visual estilo *chat app* moderno usando una paleta **azul oscuro + cian**: fondos en tonos "navy", burbujas diferenciadas para usuario y bot, gradientes suaves y sombras para profundidad. Incluye un encabezado translucido con efecto *blur*, un botón de micrófono redondo con animaciones de hover/listening y un indicador animado tipo "pulso" para mostrar el estado del sistema. También estiliza la barra de desplazamiento y garantiza una presentación responsiva en pantallas pequeñas



En conclusión, este asistente por voz tiene una **arquitectura ideal para escalar**, ya que su lógica está organizada por estados, palabras clave y módulos claros tanto en el reconocimiento de voz como en la generación de respuestas, lo que permite ampliar fácilmente nuevas áreas, necesidades, rutas conversacionales o incluso integrar fuentes externas de información sin rehacer la estructura central. Su frontend ligero y modular, junto con el uso de STT y TTS nativos del navegador, permite que el sistema crezca hacia flujos más complejos, integración con APIs, personalización de respuestas, persistencia de conversación o incluso un modelo híbrido donde convivan reglas y modelos de IA más avanzados, garantizando que el prototipo pueda evolucionar hacia un asistente académico mucho más completo y robusto.



Código HTML.

```
<!DOCTYPE html>
<html lang="es" data-theme="dark-blue">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width,initial-scale=1" />
  <title>Asistente por Voz</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <main class="app">
    <header class="app-header">
      <div class="app-title">
        <span class="dot"></span>
        <h1>De la Rosa Valdés itztli Fernanda
          & Bazaldua Montoya Jose Juan
</h1>
      </div>
      <div class="status" id="status">Inactivo — di “Escuela”</div>
    </header>

    <section class="chat" id="chat">
      <!-- Mensaje inicial -->
      <div class="msg bot">
        <div class="bubble">¡Hola! Pulsa el micrófono y di
<strong>“Escuela”</strong> para activar.</div>
        <span class="meta">Sistema</span>
      </div>
    </section>

    <footer class="composer">
      <div class="hint" id="hint">Listo para escuchar…</div>
      <div class="controls">
        <button class="mic" id="listen-button" aria-label="Escuchar"
title="Escuchar">
          <svg viewBox="0 0 24 24" aria-hidden="true">
            <path d="M12 14a3 3 0 0 0 3-3V6a3 3 0 1 0-6 0v5a3 3 0 0 0 3
3Zm5-3a5 5 0 0 1-10 0H5a7 7 0 0 0 14 0h-2Zm-5 8a7.002 7.002 0 0 1-6.32-4H8a5
5 0 0 0 8 0h2.32A7.002 7.002 0 0 1 12 19Z"/>
          </svg>
        </button>
      </div>
    </footer>
```

```
  </main>

  <script src="app.js"></script>
</body>
</html>
```

Código CSS.

```
/* ====== Tema Azul Oscuro + Cian (alto contraste) ====== */
:root {
  /* Fondo general (azules muy oscuros) */
  --bg-0: #0b1220; /* base (casi negro, tinte azul) */
  --bg-1: #0f1a2b; /* panel / header */
  --bg-2: #12233a; /* burbuja bot */
  --bg-3: #0a2540; /* acento oscuro para degradados */

  /* Texto (contraste AAA sobre bg-0/bg-1) */
  --text-0: #f2f6fb; /* principal */
  --text-1: #b7c4d6; /* suave */

  /* Acentos (fríos) */
  --accent:   #00d1ff; /* cian eléctrico principal */
  --accent-2: #4f9cff; /* azul claro de apoyo */
  --accent-3: #2dd4bf; /* turquesa (para estados/hover) */

  /* Efectos */
  --ring:   rgba(0, 209, 255, .30); /* halo cian */
  --shadow: rgba(0, 7, 20, .40);

  /* Estados */
  --mic-on: #84ccff; /* luz/estado activo */
}

* { box-sizing: border-box; }
html, body { height: 100%; }

body {
  margin: 0;
  color: var(--text-0);
  font: 15px/1.45 system-ui, -apple-system, Segoe UI, Roboto, Arial, sans-
serif;
  /* Gradientes azules (sin morado) */
  background:
    radial-gradient(1100px 600px at 85% -10%, #0a2540 0%, rgba(11,18,32,0)
45%),
```

```
    radial-gradient(900px 500px at 10% 0%,  #103054 0%, rgba(11,18,32,0)
50%),
    linear-gradient(180deg, #0e1a2e 0%, var(--bg-0) 60%);
}

/* ====== Contenedor App ====== */
.app {
  max-width: 760px;
  margin: 0 auto;
  min-height: 100dvh;
  display: grid;
  grid-template-rows: auto 1fr auto;
}

.app-header {
  position: sticky;
  top: 0;
  z-index: 5;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  padding: 16px 18px;
  background: linear-gradient(180deg, rgba(15,26,43,.95),
rgba(15,26,43,.7));
  backdrop-filter: blur(6px);
  border-bottom: 1px solid color-mix(in oklab, var(--accent) 35%,
transparent);
}

.app-title {
  display: flex;
  align-items: center;
  gap: 10px;
}

.app-title h1 {
  font-size: 16px;
  margin: 0;
  font-weight: 600;
  letter-spacing: .2px;
}

.app-header .status {
  color: var(--text-1);
```

```
  font-size: 13px;
}

/* Puntito animado (cian-azul) */
.dot {
  inline-size: 10px;
  block-size: 10px;
  background: linear-gradient(180deg, var(--accent), #1294ff);
  border-radius: 50%;
  box-shadow: 0 0 0 0 var(--ring);
  animation: pulse 2s infinite;
}

@keyframes pulse {
  0%   { box-shadow: 0 0 0 0 var(--ring); }
  70%  { box-shadow: 0 0 0 10px rgba(0,209,255,0); }
  100% { box-shadow: 0 0 0 0 rgba(0,209,255,0); }
}

/* ====== Área de chat ====== */
.chat {
  padding: 18px 16px 8px;
  overflow: auto;
  scroll-behavior: smooth;
}

/* Mensajes */
.msg {
  display: grid;
  gap: 6px;
  margin: 8px 0 14px;
  max-width: 88%;
}

.msg .bubble {
  padding: 12px 14px;
  border-radius: 14px;
  box-shadow: 0 6px 24px -10px var(--shadow);
  word-wrap: break-word;
  white-space: pre-wrap;
  border: 1px solid rgba(255,255,255,.06);
}

/* Bot (izquierda): azul profundo con leve degradado */
.msg.bot { justify-items: start; }
```

```
.msg.bot .bubble {
  color: #e7edf6;
  background: linear-gradient(180deg, var(--bg-2), #0e2a4a);
  border-color: color-mix(in oklab, var(--accent-2) 28%, transparent);
}

/* Usuario (derecha): acento cian/azul brillante */
.msg.user { justify-items: end; margin-left: auto; }
.msg.user .bubble {
  color: #071421; /* texto oscuro sobre acento claro */
  background: linear-gradient(180deg, #00d1ff, #1294ff);
  border-color: rgba(255,255,255,.10);
  /* toque de relieve interior */
  box-shadow:
    0 6px 24px -10px var(--shadow),
    inset 0 -1px 0 0 color-mix(in oklab, #ffffff 20%, var(--accent-2));
}

/* Meta (nombre/hora) */
.msg .meta {
  font-size: 12px;
  color: var(--text-1);
}

/* ====== Composer / Footer ====== */
.composer {
  position: sticky;
  bottom: 0;
  padding: 12px 16px 18px;
  background:
    linear-gradient(180deg, rgba(15,26,43,.12), rgba(11,18,32,.9) 30%, var(-
-bg-0) 100%);
  border-top: 1px solid color-mix(in oklab, var(--accent) 35%, transparent);
  display: grid;
  gap: 10px;
}

.hint {
  color: var(--text-1);
  font-size: 13px;
  padding-left: 2px;
}

.controls {
  display: flex;
```

```
  align-items: center;
  gap: 10px;
}

/* Botón micrófono: cian con brillo azul */
.mic {
  inline-size: 56px;
  block-size: 56px;
  border-radius: 50%;
  border: 1px solid color-mix(in oklab, var(--accent) 35%, transparent);
  background:
    radial-gradient(100% 100% at 70% 0%, #00d1ff 0%, #1294ff 100%);
  display: grid;
  place-items: center;
  color: #06131f; /* texto/ícono oscuro sobre acento claro */
  cursor: pointer;
  transition: transform .08s ease, box-shadow .2s ease, filter .2s ease;
  box-shadow:
    0 10px 24px -10px var(--ring),
    inset 0 0 0 1px rgba(255,255,255,.06);
}

.mic:hover {
  transform: translateY(-1px);
  filter: saturate(1.06);
  box-shadow:
    0 16px 36px -14px var(--ring),
    inset 0 0 0 1px rgba(255,255,255,.12);
}

.mic:active { transform: translateY(0); }
.mic svg { width: 26px; height: 26px; fill: currentColor; }

.mic.listening {
  outline: 2px solid var(--mic-on);
  box-shadow:
    0 0 0 6px rgba(132,204,255,.15),
    inset 0 0 0 1px rgba(255,255,255,.15);
}

/* ====== Scrollbar (cian/azul) ====== */
.chat::-webkit-scrollbar { width: 10px; }
.chat::-webkit-scrollbar-thumb {
  background: linear-gradient(180deg, #1294ff, #00d1ff);
  border: 2px solid transparent;
```

```
  background-clip: padding-box;
  border-radius: 10px;
}
.chat::-webkit-scrollbar-track { background: transparent; }

/* ====== Responsive ====== */
@media (max-width: 560px) {
  .app { max-width: 100%; }
  .msg { max-width: 95%; }
  .mic { inline-size: 54px; block-size: 54px; }
}
```

Código JS.

```
// ======== Referencias UI (Interfaz de Usuario) ========
const chatEl   = document.getElementById('chat');
const statusEl = document.getElementById('status');
const hintEl   = document.getElementById('hint');
const micBtn   = document.getElementById('listen-button');
let esperandoTareaInformatica = false; // Variable de estado para esperar la
respuesta
// de qué tarea de informática necesita el usuario
// ======== Utilidades de UI (chat) ========
function appendMessage(side, text) {
  const wrap = document.createElement('div');
  wrap.className = `msg ${side}`;
  const bubble = document.createElement('div');
  bubble.className = 'bubble';
  bubble.textContent = text;
  const meta = document.createElement('span');
  meta.className = 'meta';
  meta.textContent = side === 'user' ? 'Tú' : 'Sistema';
  wrap.appendChild(bubble); wrap.appendChild(meta);
  chatEl.appendChild(wrap);
  chatEl.scrollTo({ top: chatEl.scrollHeight, behavior: 'smooth' });
}
const setHeardText = (t) => hintEl.textContent = t || 'Listo para
escuchar…';
const setStatus = (t) => statusEl.textContent = t;

/*
Normalizamos el texto para facilitar la detección de palabras clave.
Convierte el texto a minúsculas y le quita acentos (diacríticos).
Uso de try/catch por compatibilidad:
si el motor JS no soporta la clase  Unicode
```

```
\p{Diacritic}, aplicamos un fallback clásico con rango Unicode.
*/

function normalizeSafe(s='') {
  const lower = s.toLowerCase().normalize('NFD');
  try { return lower.replace(/\p{Diacritic}/gu, '').trim(); }
  catch { return lower.replace(/[\u0300-\u036f]/g, '').trim(); }
}

// ======== Definimos AREAS y NECESIDADES
// Con esto Dividimos las distintas areas que van ligadas a una necesidad
========
const AREAS = {
  electronica: ['electronica','electrica','circuitos'],
  informatica: ['informatica','sistemas','computacion','programacion'],
  comunicaciones: ['comunicaciones','redes','telecomunicaciones'],
};
const NEEDS = ['tarea','practica','proyecto','examen','investigacion'];

// ======== Estado conversacional ========
const STATES = { IDLE:'IDLE', AWAKE:'AWAKE', AREA:'AREA' };
let state = STATES.IDLE;
let selectedArea = null;

// ======== Reconocimiento ========
const SpeechRecognition = window.SpeechRecognition ||
window.webkitSpeechRecognition;
let recognition = null;

function ensureRecognition() {
  if (!SpeechRecognition) {
    appendMessage('bot', 'Tu navegador no soporta reconocimiento de voz. Usa
Chrome sobre HTTPS o localhost.');
    return null;
  }
  const rec = new SpeechRecognition();
  rec.lang = 'es-MX';
  rec.continuous = false;
  rec.interimResults = false;
  return rec;
}

function startListenOnce() {
  recognition = ensureRecognition();
  if (!recognition) return;
```

```
  setStatus(state === STATES.IDLE ? 'Escuchando (di “Escuela”)' :
'Escuchando…');
  setHeardText('Escuchando… habla ahora');
  micBtn.classList.add('listening');

  let huboResultado = false;

  recognition.onresult = (ev) => {
    huboResultado = true;
    const last = ev.results.length - 1;
    const transcript = ev.results[last][0].transcript || '';
    const norm = normalizeSafe(transcript);

    appendMessage('user', transcript);
    setHeardText(`Reconocido: "${transcript}"`);
    handleUtterance(norm);
  };

  recognition.onnomatch = () => {
    if (!huboResultado) appendMessage('bot','No entendí lo que dijiste.');
  };

  recognition.onerror = (e) => {
    console.warn('Speech error:', e);
    const map = {
      'no-speech':'No se detectó voz.',
      'audio-capture':'No hay audio o micrófono.',
      'not-allowed':'Permiso de micrófono denegado.',
      'aborted':'Reconocimiento cancelado.',
      'network':'Error de red del servicio de voz.'
    };
    appendMessage('bot', map[e.error] || `Error de reconocimiento:
${e.error}`);
    setStatus('Error / Pausado');
  };

  recognition.onend = () => {
    micBtn.classList.remove('listening');
    if (!huboResultado && hintEl.textContent.includes('Escuchando…')) {
      appendMessage('bot','Voz no detectada.');
    }
    setHeardText('Listo para escuchar…');
    setStatus('Inactivo — di “Escuela”');
  };
```

```
  try { recognition.start(); } catch(e) { console.warn('start() ya fue
llamado', e); }
}

// ======== Lógica conversacional ========
function detectArea(norm) {
  for (const [area, variants] of Object.entries(AREAS)) {
    if (variants.some(v => norm.includes(v))) return area;
  }
  return null;
}
function detectNeed(norm) {
  return NEEDS.find(w => norm.includes(w)) || null;
}

function handleUtterance(norm) {

// =======================
//  SEGUNDA RESPUESTA
// =======================
if (esperandoTareaInformatica) {

    if (norm.includes('programacion')) {
        const respuesta = 'Programación es el proceso de crear instrucciones
para que una computadora ejecute tareas. Suelen usarse lenguajes como C,
Python o Java.';
        appendMessage('bot', respuesta);
        speakOut(respuesta);
    }

    else if (norm.includes('redes')) {
        const respuesta = 'Las redes se encargan de conectar computadoras
para compartir información. Involucran protocolos como TCP/IP y equipos como
routers.';
        appendMessage('bot', respuesta);
        speakOut(respuesta);
    }

    else if (norm.includes('bases de datos') || norm.includes('bd')) {
        const respuesta = 'Una base de datos es un sistema que almacena y
organiza información. Se maneja con SQL para realizar consultas y
administración de datos.';
        appendMessage('bot', respuesta);
        speakOut(respuesta);
```

```
    }

    else {
        const respuesta = 'No entendí tu tipo de tarea. Prueba con:
programación, redes o bases de datos.';
        appendMessage('bot', respuesta);
        speakOut(respuesta);

        // O puedes NO limpiar la variable para seguir preguntando.
    }

    // Limpiamos la espera para no repetir
    esperandoTareaInformatica = false;
    return; // <- muy importante regresar a la función principal
    // para no procesar más lógica de estado en esta iteración
}

  // Comandos globales
  if (norm.includes('cancelar') || norm.includes('terminar') ||
norm.includes('salir')) {
    state = STATES.IDLE; selectedArea = null;
    appendMessage('bot','De acuerdo, finalizamos. Di “Escuela” para volver a
empezar.');
     speakOut('De acuerdo, finalizamos. Di “Escuela” para volver a
empezar.');
    return;
  }

  if (state === STATES.IDLE) {
    if (norm.includes('escuela')) {
      state = STATES.AWAKE;
      const text = '¿En qué área ocupas ayuda? (electrónica o informática)';
      appendMessage('bot', text);
      speakOut(text);
      setStatus('Activo — esperando área');
    } else {
      const text = 'Di “Escuela” para activar.';
      appendMessage('bot', text);
      speakOut(text);
    }
    return;
  }

  if (state === STATES.AWAKE) {
    const area = detectArea(norm);
```

```
    if (area) {
      selectedArea = area;
      state = STATES.AREA;
      const text = `Área ${area} seleccionada. ¿Qué necesitas? (tarea,
práctica o proyecto)`;
      appendMessage('bot', text);
      speakOut(text);
      setStatus(`Área: ${area} — esperando necesidad`);
    } else {
      const text = 'No detecté el área. Opciones: electrónica , informática,
comunicaciones.';
      appendMessage('bot', text);
      speakOut(text);
    }
    return;
  }

  if (state === STATES.AREA) {
    const need = detectNeed(norm);
    if (need) {
      const text = `Perfecto. Preparando recursos de ${selectedArea} para tu
${need}.`;
      appendMessage('bot', text);
      speakOut(text);
              // ======== RESPUESTAS POR ÁREA + NECESIDAD:
              //  Aqui codificamos las respuestas
              // -BDs, URL, Respuesta Texto, etc========

      //Tareas
       if (selectedArea === 'informatica' && need === 'tarea'){
            const msg = '¿Que tarea necesitas?';
            appendMessage('bot', msg);
            speakOut(msg);
        //Cambiamos el estado para esperar la respuesta de qué tarea de
informática necesita el usuario
            esperandoTareaInformatica = true;
            return;
       }


       if (selectedArea === 'comunicaciones' && need === 'tarea'){
            const msg = '¿Que tarea necesitas?';
            appendMessage('bot', msg);
            speakOut(msg);
       }
```

```
        if (selectedArea === 'electronica' && need === 'tarea'){
            const msg = '¿Que tarea necesitas?';
            appendMessage('bot', msg);
            speakOut(msg);
        }
        // Practicas.
      if (selectedArea === 'informatica' && need === 'practica'){
            const msg = '¿Que práctica necesitas?';
            appendMessage('bot', msg);
            speakOut(msg);
      }
       if (selectedArea === 'comunicaciones' && need === 'practica'){
            const msg = '¿Que práctica necesitas?';
            appendMessage('bot', msg);
            speakOut(msg);
       }
        if (selectedArea === 'electronica' && need === 'practica') {
            const msg = '¿Que práctica necesitas?';
            appendMessage('bot', msg);
            speakOut(msg);
       }
        //Proyectos
      if (selectedArea === 'informatica' && need === 'proyecto'){
            const msg = '¿Que proyecto necesitas?';
            appendMessage('bot', msg);
            speakOut(msg);
       }
       if (selectedArea === 'comunicaciones' && need === 'proyecto'){
            const msg = '¿Que proyecto necesitas?';
            appendMessage('bot', msg);
            speakOut(msg);
       }
        if (selectedArea === 'electronica' && need === 'proyecto'){
            const msg = '¿Que proyecto necesitas?';
            appendMessage('bot', msg);
            speakOut(msg);
       }


        setTimeout(() => {
        const done = 'Listo. Si ocupas algo más, di “Escuela” para volver a
activar.';
        appendMessage('bot', done);
       speakOut(done);
```

```
        state = STATES.IDLE; selectedArea = null; setStatus('Inactivo — di
“Escuela”');
      }, 900);
    } else {
      const text = '¿Es una tarea, práctica o proyecto?';
      appendMessage('bot', text);
      speakOut(text);
    }
    return;
  }
}

// ======== (Opcional) Hablar en voz alta ========
function speakOut(text) {
  if (!('speechSynthesis' in window)) return;
  const u = new SpeechSynthesisUtterance(text);
  u.lang = 'es-MX';
  window.speechSynthesis.cancel();
  window.speechSynthesis.speak(u);
}

// ======== Eventos ========
micBtn.addEventListener('click', startListenOnce);
```